

RA2 Report

Sergio Fabregat López

May 20, 2026

1 Overview

The main features of this Vulkan based graphics engine are:

- Deferred Rendering.
- **Physically Based Shading**, based on Unreal's shading model.
- **Screen Space Ambient Occlusion**.
- **FXAA**.
- Shadows, implemented in two different ways: **Variance Shadow Maps (VSM)** and **Ray Traced Soft Shadows**.
- A moving camera.
- A user interface to tweak the shaders' parameters.

In order to achieve these effects our engine has the following render pass architecture:

1. **Depth Pre-Pass:** We compute depth first so the deferred pass can take advantage of early Z rejection for all fragments.
2. **Shadow Map Pass:** We write the shadow map for each light in the scene.
3. **Shadow Map Blur Pass:** Since we're using VSM we can directly blur the shadow map to obtain smooth shadows.
4. **Deferred Pass:** We write the G buffer textures (albedo, position, normals, metallic/roughness and material ID) needed for the later passes.
5. **Ray Traced Shadows Pass:** We compute visibility using simple Monte Carlo ray tracing.
6. **Ray Traced Shadows Blur Pass:** We smooth over the noisy result from the previous pass.
7. **SSAO Pass:** We compute Screen Space Ambient Occlusion.
8. **SSAO Blur Pass:** We smooth over the noisy result from the previous pass.
9. **Composition Pass:** We take all the textures computed in the previous passes (G Buffer, shadow maps, smoothed SSAO and smoothed ray traced visibility) and put it all together to compute the scene's lighting.
10. **Post Process Pass:** We apply antialiasing to the composed image via FXAA and draw the UI over it using ImGui.

2 Physically Based Shading

Our Physically Based Shading model is based on Unreal’s shading model. We use only three parameters to control this simple model, **albedo**, **roughness** and **metallic**. We implement a BRDF in which the diffuse component is a regular Lambertian, and the specular component is modeled using Cook-Torrance’s microfacet model:

$$f_s(\mathbf{l}, \mathbf{v}) = \frac{D(\mathbf{h})F(\mathbf{v}, \mathbf{h})G(\mathbf{l}, \mathbf{v}, \mathbf{h})}{4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}$$

First, we compute the normal distribution function, D , using the GGX distribution. The roughness parameter is transformed to α by squaring it, and then used to compute D as follows:

$$D(\mathbf{h}) = \frac{\alpha^2}{\pi((\mathbf{n} \cdot \mathbf{h})^2(\alpha^2 - 1) + 1)^2}$$

Next, we compute the geometric term G . In this case, we use *Schlick*’s separable approximation.

$$G_1(\mathbf{v}) = \frac{\mathbf{n} \cdot \mathbf{v}}{(\mathbf{n} \cdot \mathbf{v})(1 - k) + k}$$

$$G(\mathbf{l}, \mathbf{v}, \mathbf{h}) = G_1(\mathbf{l})G_1(\mathbf{v})$$

We compute k the following way, as an adjustment made for analytic lights:

$$k = \frac{(\text{Roughness} + 1)^2}{8}$$

Then, we compute the Fresnel term (F). In this case, we use Schlick’s approximation, but with one modification, a Spherical Gaussian is used to replace the power function:

$$F(\mathbf{v}, \mathbf{h}) = F_0 + (1 - F_0) 2^{(-5.55473(\mathbf{v} \cdot \mathbf{h}) - 6.98316)(\mathbf{v} \cdot \mathbf{h})}$$

F_0 is computed by linearly interpolating between a regular dielectric’s reflectance, 0.04, and the material’s albedo based on the metallic value. This is because metallic materials tint their specular reflection towards their base color.

Lastly, we mix the diffuse and specular lobes using k_d as a coefficient to scale the diffuse component in order to maintain energy conservation:

$$f(\mathbf{l}, \mathbf{v}) = k_d f_d + f_s(\mathbf{l}, \mathbf{v})$$

F represents the proportion of light that is reflected off the surface. As such $1 - F$ represents how much light is refracted into the surface. Since the diffuse component models light that enters the surface and is scattered internally before exiting, we use the fraction of light not reflected by the Fresnel term to scale the diffuse contribution. Additionally, we also use the complement of the metallic parameter to compute k_d , because metals should have no diffuse component, as they reflect all the light that comes into contact with them.

$$k_d = (1 - \max(F(\mathbf{v}, \mathbf{h}))(1 - \text{metallic}))$$

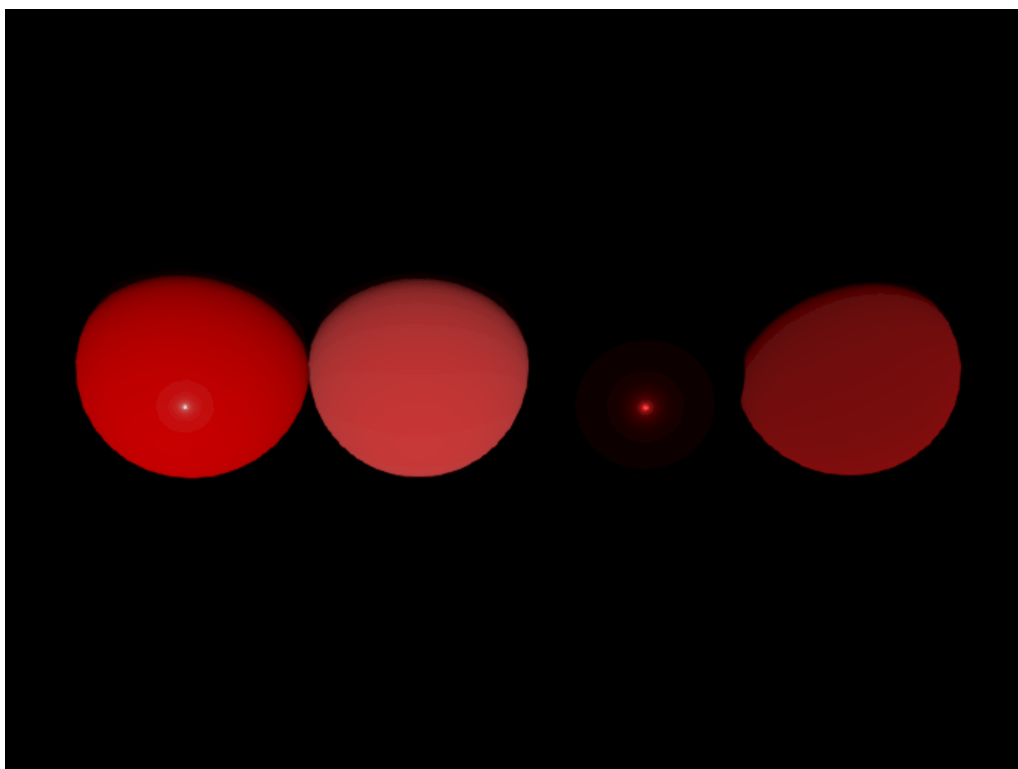


Figure 1: Our shading model in action. The spheres all share the same albedo. However, from left to right, the first one has very low roughness and no metallic, the second one has maximum roughness and no metallic, the third one has low roughness and maximum metallic and the last one has maximum roughness and maximum metallic.

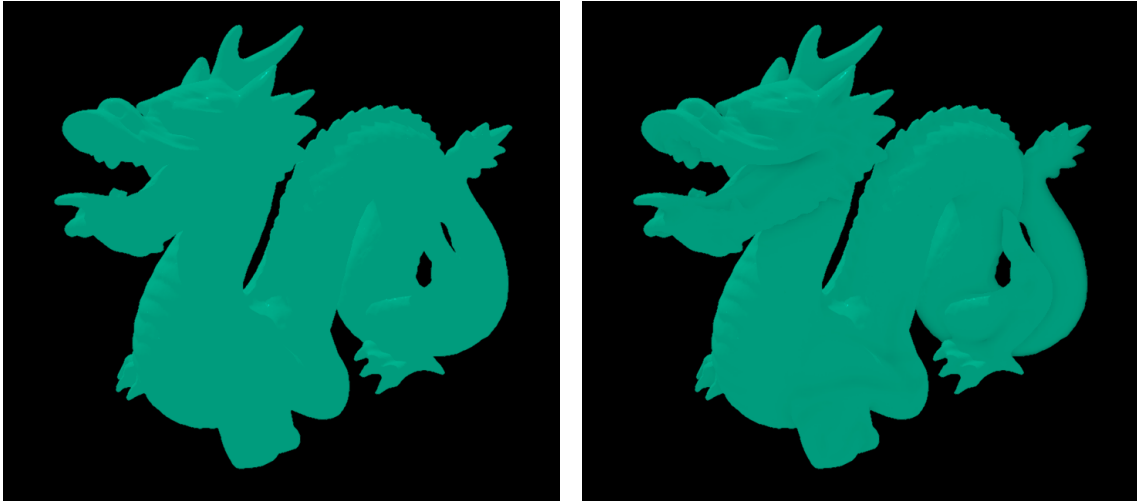


Figure 2: The Stanford Dragon without SSAO on the left, and with SSAO on the right. It can be observed how SSAO adds detail to areas that aren't directly lit.

3 Screen Space Ambient Occlusion

The implementation of Screen Space Ambient Occlusion (SSAO) is done in two passes, as previously illustrated. The following implementation of the first pass is based on [LearnOpenGL's SSAO tutorial](#).

The first pass computes the occlusion factor at each fragment. In order to do that we take multiple depth samples in a hemisphere sample kernel surrounding the fragment position, and compare each of the samples with the current fragment's depth value. The samples that have a higher depth value than the fragment's depth are occluded, and the amount of these occluded samples is directly proportional to the occlusion factor.

It's important to note that we also added a range check in order to stop objects that are far away from contributing to the occlusion factor when computing the occlusion of a fragment near the edge of a surface.

However, each fragment's kernel can't be the same as every other fragment's, because that would cause visible patterns and artifacts. And many different kernels would take up a lot of memory, since each kernel is made up of a high number (50+) of 3D vector samples.

The solution is to generate one kernel, and jitter it randomly using a noise texture tiled across the screen. In order to place the hemisphere sample kernel facing in the direction of the fragment's normal, we need to build an orthonormal basis using said fragment's normal. This is where the noise texture comes in. Each texel contains a different random 2D vector which is used to rotate the orthonormal basis. This ensures neighboring fragments take different depth samples.

However, as we can see in [Figure 3](#), there are still visible patterns and noise in the resulting occlusion factors. In order to alleviate that we will smooth the result in a separate pass using a bilateral filter. Here is where we deviate from LearnOpenGL's implementation.

The second pass runs a bilateral filter that uses depth and normal information to blur the SSAO while preserving the geometry's edges. The size of the blur's kernel is the same as the size of the noise texture. Because this texture is tiled across the screen, it introduces high-frequency noise with a specific repeating period. If the blur kernel were smaller than this period, it would fail to capture and average out the full noise pattern, leaving those visible grid-like artifacts still on the screen.

Finally, each fragment's occlusion factor is used when computing the light's ambient component. The occlusion factor scales ambient light, adding more detail to the simulation of indirect lighting. The result of the SSAO blur pass can be seen in [Figure 4](#)



Figure 3: The Stanford Dragon's head occlusion factors after the first SSAO pass.



Figure 4: The Stanford Dragon's head occlusion factors after the SSAO blur pass.

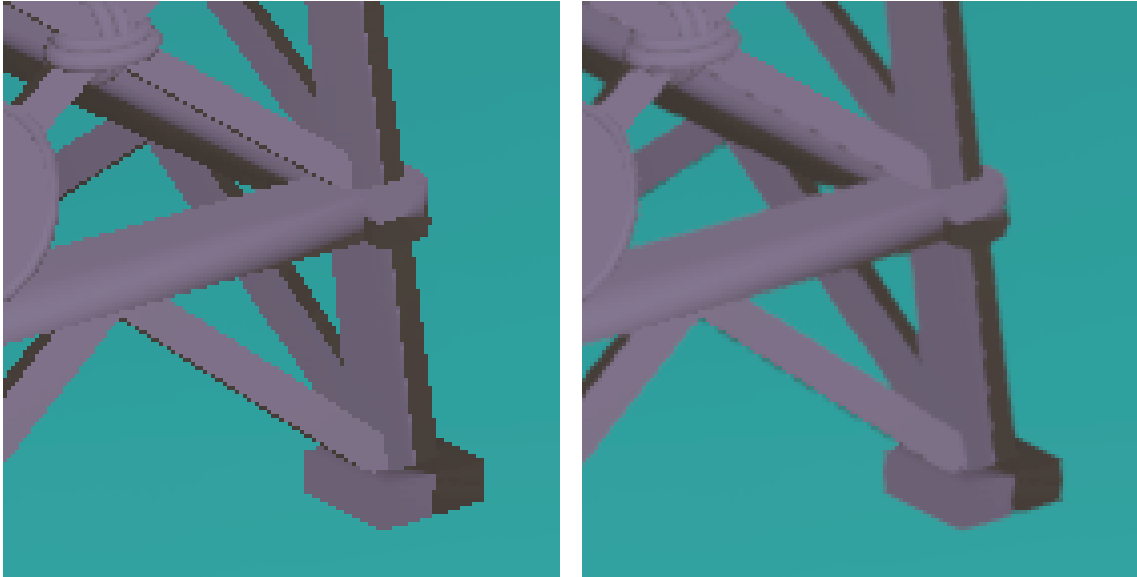


Figure 5: A watchtower's base. Without any anti-aliasing applied on the left, with FXAA on the right.

4 FXAA

Anti-aliasing is used to smooth over the jagged edges caused by having a discrete set of points where we sample the scene (the pixels), and the fact that most edges aren't straight horizontal or vertical lines (Figure 5). There are many algorithms to achieve anti-aliasing, we implemented one of the fastest, FXAA.

The following implementation is based on [Simon Rodriguez's FXAA tutorial](#). The algorithm works by detecting high-contrast edges in the rendered image and selectively blurring them to eliminate the jagged, pixelated artifacts.

The pass first determines whether the current fragment is actually part of an edge. In order to do that, it converts the color of the current fragment and its four immediate neighbors into luma. By finding the maximum and minimum luma values among the fragment and its neighbors, it computes the local contrast. If this contrast is lower than a specific threshold, the fragment is in a smooth area and the shader returns the original color to save processing time. The shader also finishes if the area is jagged but dark, as dark pixelated edges are barely noticeable, and FXAA prioritizes performance above all else.

For fragments that are part of an edge, the algorithm first ascertains the exact orientation of the edge. It does this by sampling the four diagonal corners and computing luma gradients along the horizontal and vertical axes. This establishes whether the local edge is primarily horizontal or vertical, and which side of the edge is the steepest.

Once the direction is known, the shader explores along the edge in both directions until it detects a significant change in luma. This change indicates that it has reached the end of the edge.

Finally, by comparing the distances to both ends of the edge, the algorithm calculates a precise sub-pixel offset. Additionally, it computes a secondary offset based on the average luma of the entire 3x3 neighborhood, used to alleviate sub-pixel aliasing. The maximum of these two offsets is used to offset the UVs orthogonally to the edge and sample the final, smoothed color.

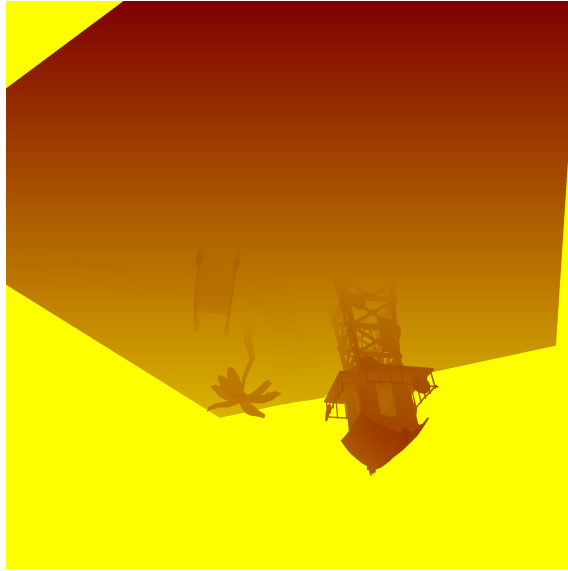


Figure 6: A point light's shadow map.

5 Variance Shadow Maps

Shadow mapping is a technique where the scene is rendered from the light's point of view and the depth buffer is stored in a texture called shadow map (Figure 6). Later, in the lighting/composition pass, we can determine which fragments are in shadow by comparing their depth against the one stored in the shadow map.

Shadow maps, however, also suffer from aliasing. In order to solve this, it's crucial to properly filter the shadow map texture. However, hardware texture filtering is inapplicable to standard shadow maps. The hardware can average the depth values contained in the shadow map, but that causes more problems than it solves, since the resulting filtered depth will still be subjected to a single binary depth comparison.

A solution to this problem are Variance Shadow Maps (VSM), which represent depth data in a manner that can be filtered linearly. The following implementation is based on this [GPU Gems article](#).

The first pass of the algorithm renders the scene from the light's point of view, but we store two values instead of one. On the red channel we store depth, and on the green channel we store depth squared. We'll later use these values to recover the mean μ and variance σ^2 of the local depth distribution.

In our specific implementation we do a geometry shader trick to not have to run a separate pass for every light. Instead we duplicate the scene's geometry in the geometry shader for each light in the scene, render the scene from each light's point of view and write the results to a 2D texture array.

Additionally, in order to solve another of the traditional issues of shadow maps, the shadow bias, we must add an addendum to the computation of second moment (depth squared) we'll use to compute the variance. To prevent surface acne on slanted polygons, we utilize hardware derivatives ($dFdx$ and $dFdy$). By adding a fraction of the squared derivatives to the second moment, we're accounting for the depth variation that occurs inside a single texel. This dynamically increases the variance based on the surface slope, absorbing floating-point inaccuracies correctly for every scene and every light, unlike a traditional constant shadow bias.

During the composition pass, we sample the VSM texture array using hardware linear filtering. Because we stored the depth and depth-squared, the hardware filtering automatically computes the mean depth and the mean of the squared depth over the sampled region. From these, we can compute σ^2 .

Then, if the current fragment's depth t is less than or equal to the mean depth, the fragment is considered fully lit. If the fragment is deeper than the mean depth, we calculate the maximum probability (P_{max}) that it is still in the light using Chebyshev's one-tailed formula. This probability is then used as the shadow attenuation factor.

$$P_{max} = \frac{\sigma^2}{\sigma^2 + (t - \mu)^2}$$

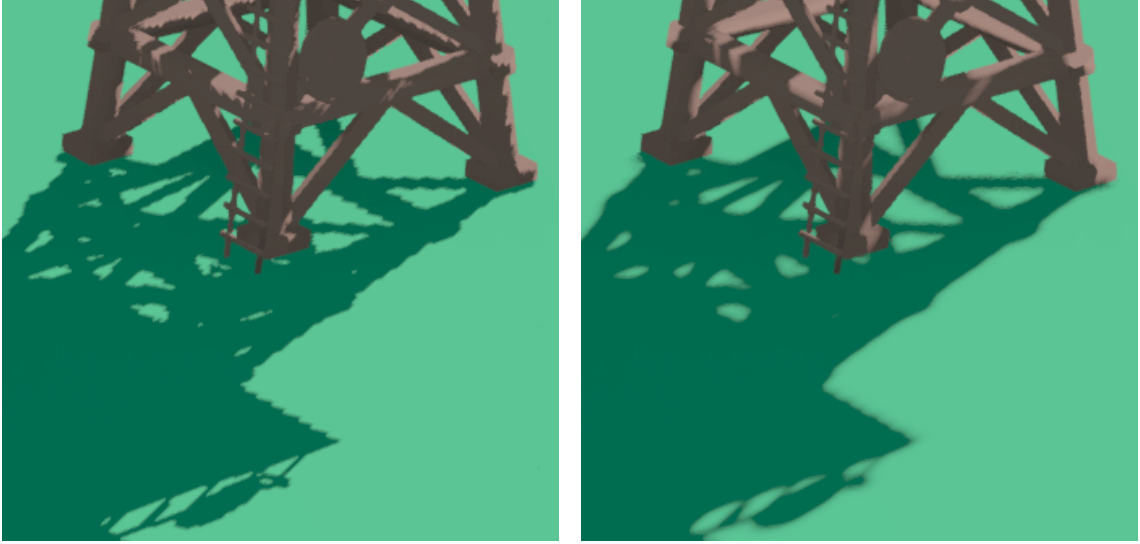


Figure 7: The watchtower’s shadow. With just hardware filtering on the left, after the gaussian blur on the right.

This probability naturally produces a smooth gradient along the edges of the geometry entirely through hardware filtering, removing aliasing. But, depending on the shadow map resolution, among other factors, hardware filtering might not be enough. In our specific case, our shadows still looked blocky and we could clearly discern the different texels in the shadow map. So, we took advantage of the fact that this representation of depth allows us to directly filter it, and added a second pass that applies a gaussian blur to the shadow map. See [Figure 7](#).

One known issue of variance shadow maps is the potential for light bleeding, light that occurs in areas that should be fully in shadow. However, there’s a way to alleviate it.

An important observation is that incorrect shadow on fully occluded surfaces will never reach full intensity. We can remove these incorrect regions by modifying P_{max} so that any values below some minimum intensity are mapped to 0, and the remaining values are rescaled so that they map from 0 (at the minimum intensity) to 1. This mostly solves the issue at the cost of some shadow detail, as we are overly darkening the penumbra regions of the shadow.

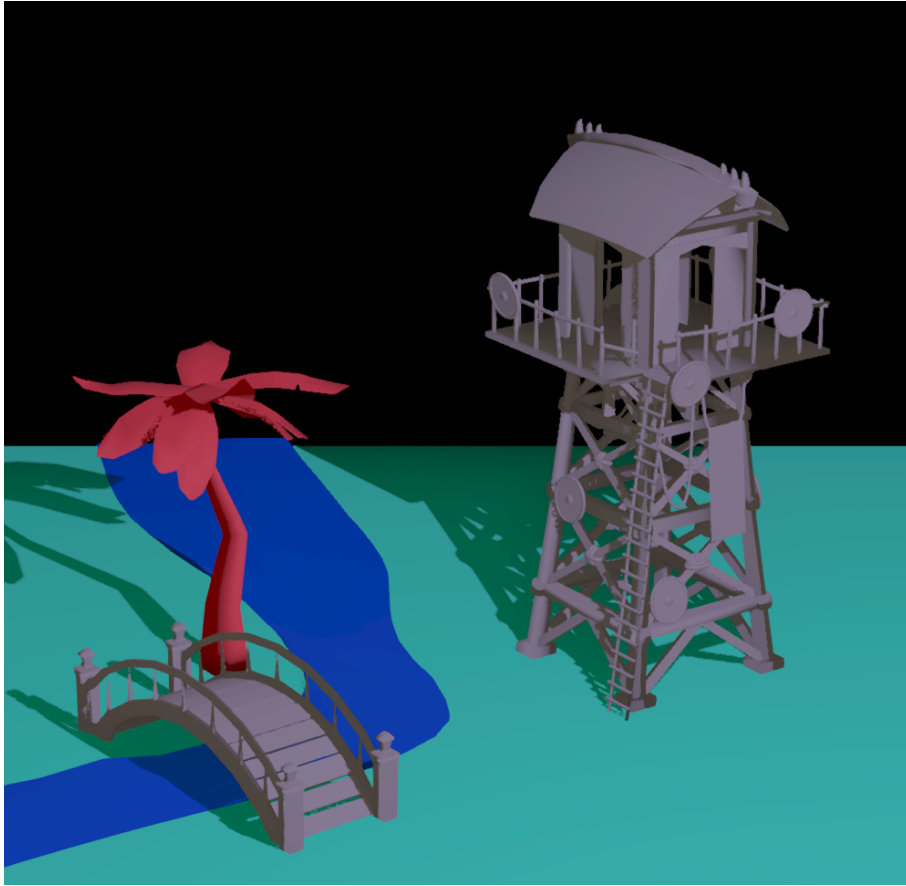


Figure 8: The result of our Variance Shadow Maps pipeline.

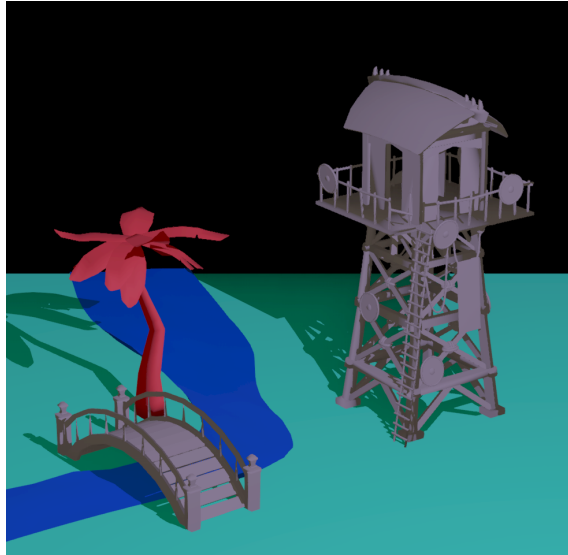


Figure 9: Hard ray traced shadows achieved by shooting a single ray in the direction of the light.

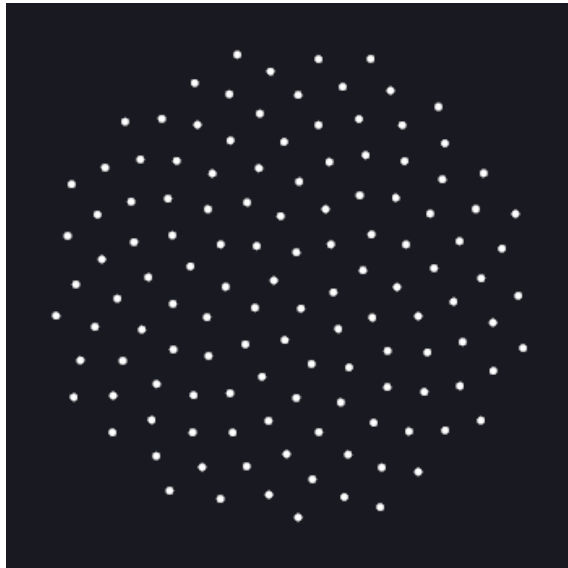


Figure 10: A Vogel disk with 128 samples.

6 Ray Traced Soft Shadows

Another more modern technique to render shadows is through ray tracing. We take advantage of the hardware-accelerated ray tracing provided by modern GPUs, and trace rays from a fragment to the light source. If we hit something that isn't the light, the fragment is in shadow. Otherwise, it's lit. However, by just shooting one ray like this we can only achieve hard shadows (Figure 9).

In order to achieve realistic soft shadows that naturally diffuse as the distance from the occluder increases, we must shoot multiple rays and average their visibility to get our shadow factor, i.e, we must perform Monte Carlo integration.

The question then becomes, in which direction do we shoot these rays? In order to get soft shadows we can't shoot all rays in the same direction (the light direction $\mathbf{l}$), because all rays would return the exact same visibility result. We must perturb $\mathbf{l}$ slightly for each ray to simulate an area light.

In order to achieve this we'll make use of a [Vogel disk](#) (Figure 10). A Vogel disk generates sample points spread evenly in a circular kernel using the Golden ratio. This provides a form of stratified sampling, which reduces the variance and noise of our shadows compared to pure random sampling.

In the shader, we tile and sample a noise texture that was generated on the CPU during application initialization, similar to our SSAO implementation. This noise texture provides a



Figure 11: The palmtree’s shadow. Before the bilateral filter pass on the left, after the blur pass on the right.

random angle in the range $[0, 2\pi]$. We use this random angle to rotate the starting position of our Vogel disk, ensuring neighboring pixels don’t share the same sampling kernel.

By doing this, we can generate the Vogel disk procedurally inside the shader, computing one sample point per ray. Generating these samples in the shader incurs almost no performance penalty. Because the Vogel disk algorithm doesn’t depend on dynamic variables like fragment position or light direction, the shader compiler can unroll the loop and pre-compute the sample coordinates at compile time. The only runtime cost is applying the random per-pixel rotation to the pre-computed disk.

Before returning the averaged visibility, we apply a correction curve that allows us to darken and thicken the shadows if we wish, for aesthetic purposes.

As can be seen in [Figure 11](#), the resulting shadow factors are a little noisy. Since we didn’t have the time to implement spatio-temporal accumulation, we only run the texture through a bilateral filter that uses normal and depth information to smooth over the noise.

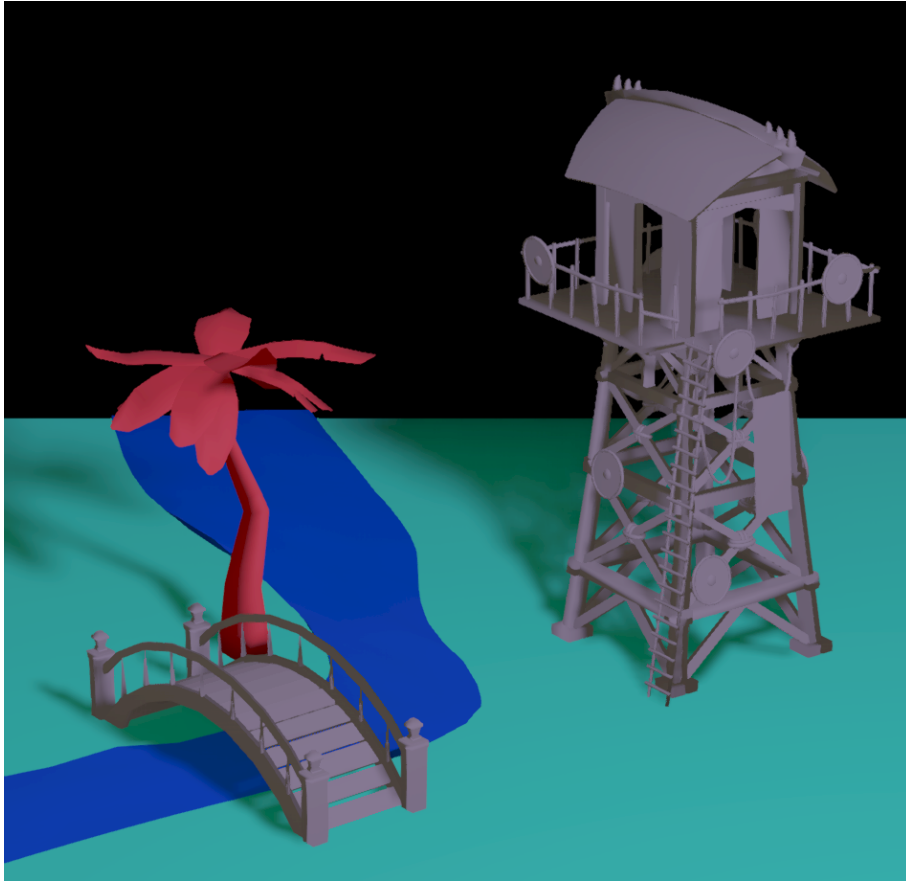


Figure 12: The result of our ray-traced shadows pipeline. This picture was taken with 16 rays per pixel, noise texture of 512x512, a ray cone radius of 0.04 and a blur kernel of 13 pixels.